Pinochle - Design Analysis Report

SWEN30006
Word Count: 2322

Written By:
Max Chivers - 1473341
Mikaël D'Agostino - 1711143
Noah Eid - 1322734

Contents:

0. Introduction 2
1. Refactoring 2
2. Additional Melds Support 3
2.1 Problem Description 3
2.2 Final Design and Rationale 3
3. Smarter Computer Bidding 4
3.1 Problem Description 4
3.2 Final Design and Rationale 4
4. Cut-throat Mode 4
4.1 Problem Description 4
4.2 Final Design and Rationale 5
5. Smart Computer Trick-Taking 6
5.1 Problem Description 6
5.2 Final Design and Rationale 6
6. Conclusion 7
7. Other Supporting Diagrams 8

0. Introduction

The development company CardCraft Studio is expanding into the digital gaming industry and aims to recreate the game of Pinochle! Though unfortunately, the initial version of the game they made suffers from poor software design choices, making it challenging to extend. We have been tasked with improving the structure and functionality of the system by implementing four key tasks.

This system must support additional melds, enable smarter bidding and trick-taking behaviour and finally we need to introduce a new cut-throat game mode. These changes aim to create a more challenging and different version of the classic Pinochle game while making sure the system remains extendable for future updates to the game.

This report presents the design and implementation of these four enhancements. For each task, we outline a solution guided by GRASP principles and Gang of Four (GoF) pattern designs while also explaining any other approach our team may have had and justifying our final design choice on the basis of what is the most flexible, modular, extendable and maintainable.

1. Refactoring

The initial state of the codebase was a mess because all logic was inside an 1000+ lined file (Pinochle.java). This breaks almost every good design principle in software, and in order to implement any of the enhancements we mentioned earlier, it needed to be refactored.The goal of refactoring was to make sure the code was modular, with low coupling, high cohesion, and single responsibilities.

At a high level, this refactored architecture (*Figure 1*) exemplifies several key GRASP principles. The Information Expert principle is demonstrated through specialized managers like *MeldManager* and *TrickManager*, which encapsulate the data and behavior specific to their domains. Low Coupling is achieved by having classes interact through well-defined interfaces rather than direct dependencies, while High Cohesion ensures each class has a focused, single purpose. The InformationExpert

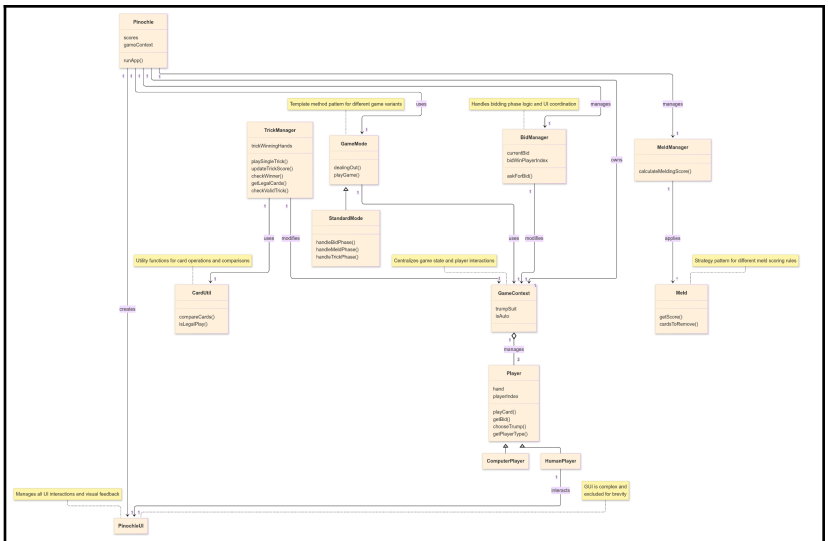

Figure 1: A domain class diagram of the initial refactored code.

and Controller pattern is evident in *GameContext* and *Pinochle*, which coordinates interactions between subsystems by containing game logic itself. Creator responsibilities are properly assigned, with *Pinochle* managing the lifecycle of managers and game components. This GRASP-compliant design eliminates the original monolithic file, making the system significantly more maintainable, testable, and extensible while reducing the ripple effects of changes across the codebase when adding on the extensions discussed below.

2. Additional Melds Support

2.1 Problem Description

In the current implementation, the game only supports a limited set of basic melds, but we would want it to support more custom melds. The system must recognise a more general range of meld types such as Dix, double runs, pinochle combinations and more. As these melds vary in card combinations and point value, we must process melds in order of descending score and make sure each card in a player's hand contributes to only one meld (to avoid any extra duplicate counts). To support further expansion, the design must allow for easy integration of new meld types without requiring major changes to the core logic.

2.2 Final Design and Rationale

We introduced a *Meld* abstract class in which we require each subclass to define the essential methods: *getScore()* and *cardsToRemove()*. Each concrete Meld is then defined as a subclass of the *Meld* abstract class and they implement those two methods accordingly, making the system much more extendable. These methods are pretty self explanatory: *getScore()* returns the value of the meld and *cardsToRemove()* return the list of cards that are used in the meld to then remove later. This applies the GRASP principles of polymorphism because we have one single *Meld* template from which all the concrete melds implement.

The *MeldManager* class was created to encapsulate all the melds and it contains a method, *calculateMeldingScore*, that loops over each Meld and calculates the score of the hand of cards you have while removing the card from each meld that was actually used. This allows for significantly lower coupling and higher cohesion, because instead of having every meld type as a function in one giant class, we break it up such that the meld classes are now focused exclusively on calculating the Meld of a hand and they all have a common parent abstract class. This makes it much simpler to have more melds with less dependencies unlike the initial design where each meld was defined as a separate method in *Pinochle* making it a big mess to understand and decipher what was

going on. The old stricture does not follow GRASP principles, as it does not use polymorphism and provides no clear abstraction for what a meld is or its role. Without a defined *Meld* type, the system lacks clarity, flexibility, and separation of responsibilities.

3. Smarter Computer Bidding

3.1 Problem Description

In the current implementation, the computer player selects bids at random without taking into account the strength of its hand or overall state of the game. This obviously results in unstrategic bidding that leads to unrealistic gameplay. The goal of this extension is to implement a smarter bidding computer that plays based on a rule based strategy to make the game a bit more challenging and not bore the human player. Based on its meld score, the computer evaluates its hand to make an opening bid. It then determines following bids by calculating the number of cards in a suit and the possible trump suit. The bidding logic must find a balance between hand strength and aggression.

3.2 Final Design and Rationale

To fix the limitations of the existing bidding logic, and lack of options for bidding of the computer, an implementation of a smarter bidding system using the Strategy Pattern was obviously needed. A new interface, *ComputerBiddingStrategy*, was written with a method *calculateBid()*, which abstracts the bidding logic. A concrete implementation, *SmartComputerBiddingStrategy*, encapsulates the rule-based decision making, basing itself on the meld score (for the opening bid) and suit distribution (for subsequent bids) to produce more realistic and strategic behaviour. The *ComputerPlayer* delegates bidding decisions to its strategy, allowing the *BiddingManager* to remain focused on flow control.

This design was selected over the alternative of embedding the bidding logic directly into *BiddingManager*. Even though that approach would be simpler for this limited rule set, it risks violating key design principles such as Single Responsibility and Open/Closed. As bidding behaviour could grow more complex, modifying *BiddingManager* would make this whole system more fragile and harder to understand for us programmers.

In contrast, the Strategy Pattern enables high cohesion by isolating bidding logic into a dedicated class, and supports extensibility by allowing different strategies to be added with minimal impact on existing code. It also clearly follows GRASP principles we

learned in class by giving the *ComputerPlayer* control over its own bidding behaviour, and encapsulates domain knowledge within well-defined components.

4. Cut-throat Mode

4.1 Problem Description

Currently we just have a standard game mode which is just the basic version of Pinochle where each player is dealt 12 cards and they take turns playing depending on the previous round. In this task we would like to extend this by adding the support for a cut-throat mode in which an additional step after bidding will be in place in gameplay (to make the game more fun and add an extra twist). The dealer begins by flipping the top two cards from the stockpile face-up and the non dealer chooses one and adds it to their hand and the dealer takes the second card. Then each player takes turns drawing their cards until each player has 24 cards and then the players will decide to keep their best 12 cards that they will use for the melding and trick taking during the game, the other cards are discarded, and then the game begins like before!

4.2 Final Design and Rationale

The original Pinochle implementation was designed around a single, standard *GameMode* implementation. To address the missing phase present in cut-throat game mode, we created a new subclass, *CutthroatMode*, which extends the existing *StandardGameMode* because we only need to add in a phase after the bidding. This extension approach was intentional, as cut-throat mode is fundamentally an extension of the standard gameplay with additional phases, allowing us to reuse and build upon the core game structure.

We also defined the *GameMode* abstract class to include key methods such as *playGame()* and a dealing method, which allows the game logic to be modular and interchangeable. This design allows the system to switch between different game modes seamlessly which improves flexibility and maintainability. Rather than isolating Cutthroat logic in separate components, we integrated the entire Cutthroat-specific process directly within the CutthroatMode class. This keeps the gameplay flow aligned with Cutthroat's unique rules while avoiding unnecessary complexity so then it improves readability later on. The playGame() method orchestrates the game through clearly defined phases: bidding, post-bid handling (specific to Cutthroat), melding, and trick-taking.

Within the *handlePostBidPhase()* method, we implemented the missing cut-throat specific logic. By embedding GUI interaction directly in the mode logic was a deliberate

choice for simplicity and a direct user experience. We are encapsulating human interaction within a popup inside the mode class and we kept the behavior self-contained and easier to understand without adding extra controller layers so then the auto part of the card choosing in the tests can be accomplished without the user clicking on cards on the screen. This design also ensures the mode class fully controls its own lifecycle, adhering to the Controller and Information Expert GRASP principles.

We considered alternatives, such as precomputing card choices via configuration files or introducing dedicated manager classes for post-bid handling, but these approaches were unnecessary given the scope. Since cut-throat mode has a limited complexity and the computer's strategy is deterministic, embedding the entire Cutthroat pipeline inside the CutthroatMode class was the most practical and maintainable solution.

5. Smart Computer Trick-Taking

5.1 Problem Description

Following in the foundation laid out in Task 2, instead of selecting cards at random or simple rules, we need to make a smarter computer player which will allow for better gameplay. To test its effectiveness, the computer player must produce results that beat the random player when they play against each other. In order to achieve this the smart player must track all previously played cards alongside its current hand and use this information to make strategic decisions in order to try and maximize its score. This must be achieved while keeping the system flexible, maintainable and follow the GRASP and GoF patterns.

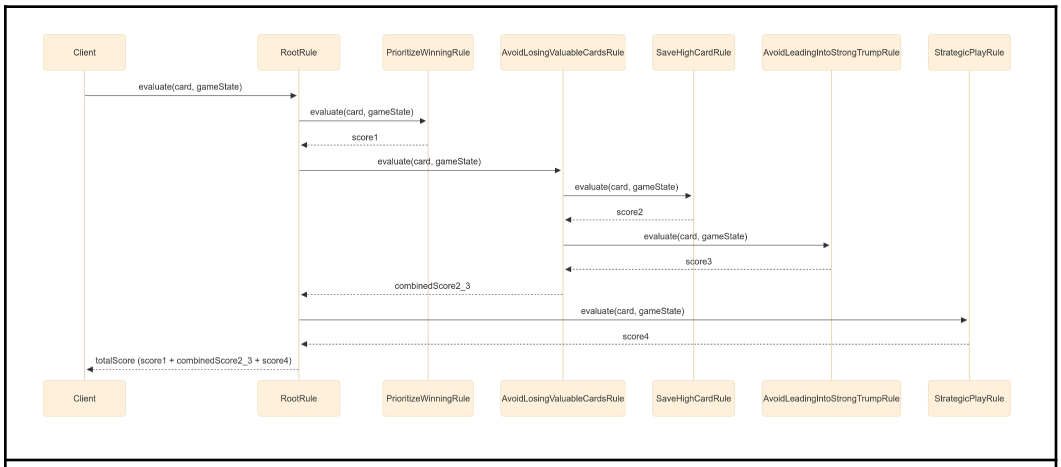

Figure 2: A sequence diagram for rules of the computer's decision making.

5.2 Final Design and Rationale

The Composite Pattern was used to implement this smart strategy because it naturally represents the hierarchical nature of strategic card game decision-making. The pattern

allows us to treat individual rules (leaf nodes) and collections of rules (composite nodes) uniformly through a common RuleComponent interface, enabling complex decision trees where high-level strategies like "avoid losing valuable cards" decompose into specific sub-rules with different weighting and combination strategies. Putting this together with the Strategy Pattern allows complex rules that the computer player follows to be changed. By passing in different rule structures into the *SmartTrickTakingStrategy* constructor which takes a *RuleComponent* (meaning this tree can be customized for different ways of playing). Currently, it is defaulted to a set *RootRule*, following a basic flow of key decisions in making better card choices for trick-taking (see Figure 2 and Figure 3). Additionally, if an entirely separate strategy would be needed, independent of the *RuleComponent*s, then it could be easily extended by implementing the interface, which the computer player utilises (see Figure 4 and Figure 5).

New rules can be seamlessly integrated into the existing tree without modifying existing code, adhering to the Open/Closed Principle. For instance, the *StrategicPlayRule* uses weighted sums to balance aggressive exploitation moves against conservative play, while the root rule simply aggregates all strategic considerations. This flexibility enables creation of distinct computer play styles by adjusting rule weights and tree composition rather than rewriting core logic.

For this task, it was considered to do a similar Strategy Pattern implementation, where some basic flow could be implemented to make a bidding strategy. The difference with this is that the trick-taking phase is a lot more complex, and less straightforward. Bidding logic is simple, and based on the Melding phase, where there is a maximum score that could be reached. Therefore, a more complex system was needed, hence the Composite Pattern, paired with Strategy Pattern.

6. Conclusion

This project helped us understand how to build and maintain scalable software and the importance of doing so. We presented how we redesigned the current game to maintain functionality but apply GRASP and GoF principles to ensure our solutions are extendable, flexible and optimized. Alongside redesigning the system, we added four new improvements: support for additional melds, a smarter computer bidding, a cut-throat mode and a smart trick taking for the computer player. There were alternative design patterns that could have been used for each task but we chose the one that offered the best clarity, maintainability and future scalability as explained throughout this report.

7. Other Supporting Diagrams

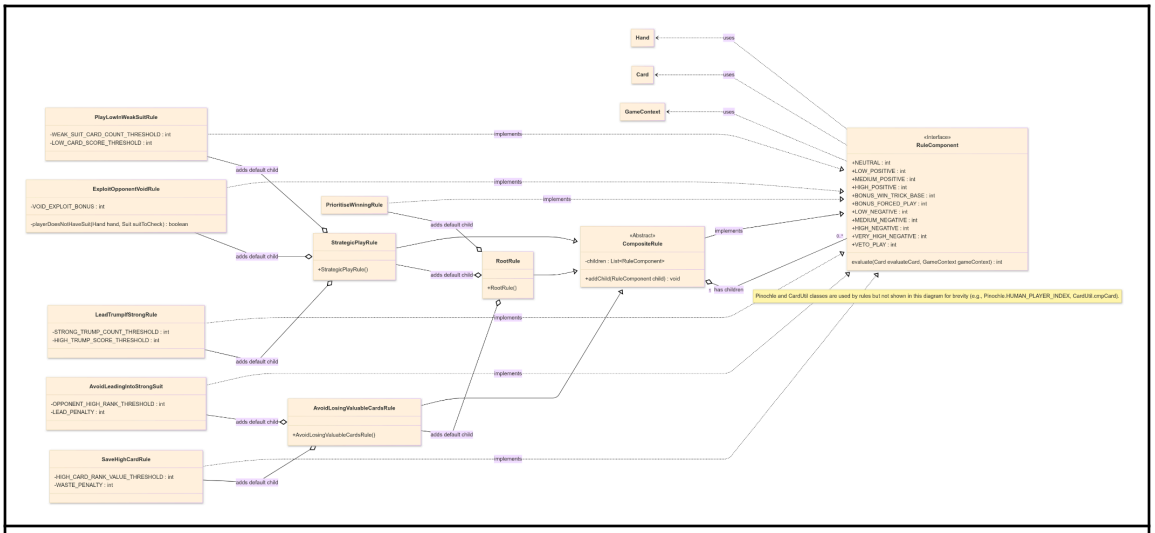

Figure 3: Design class diagram of Rules for task 4.

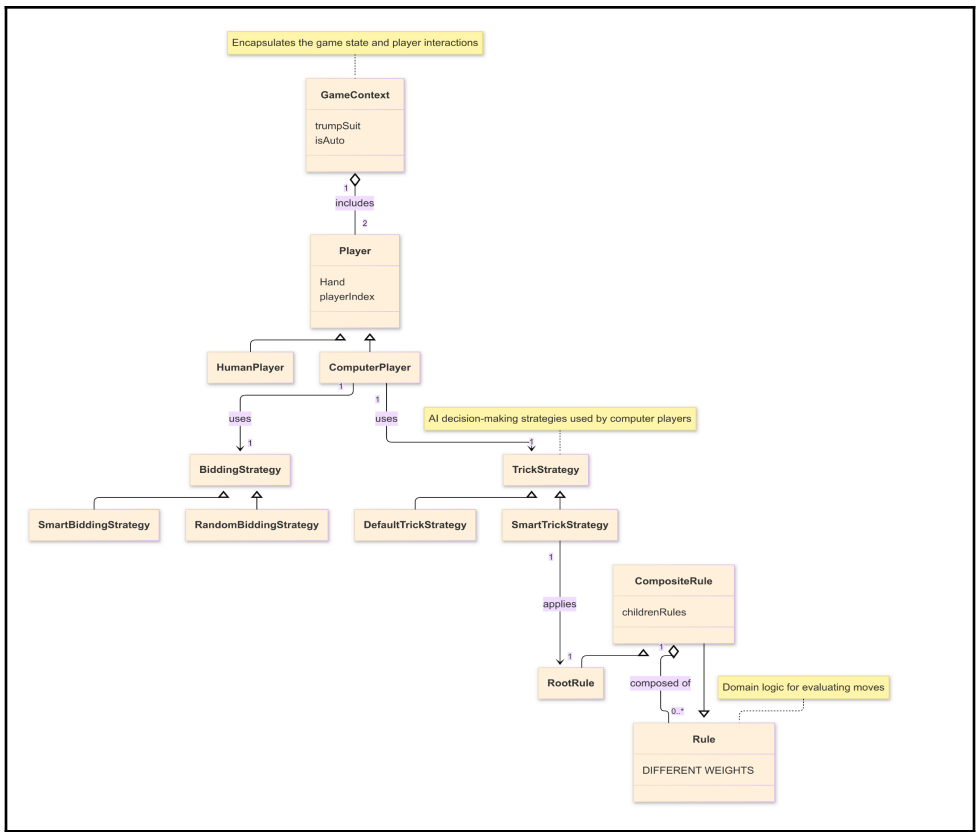

Figure 4: Domain class diagram of all Player classes utilising strategies.

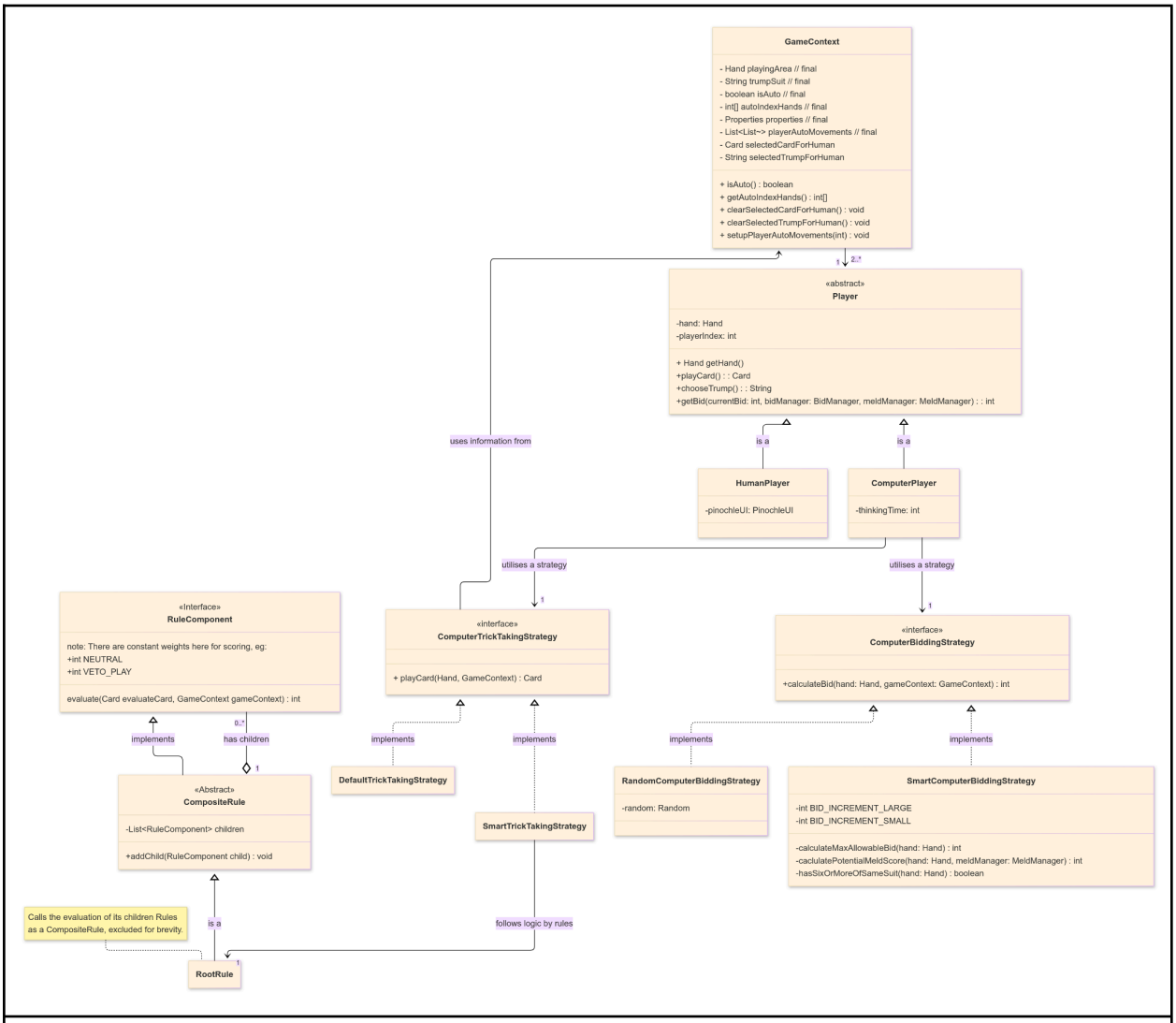

Figure 5: Design class diagram of all Player classes utilising strategies.